

CAT ASSIGNMENT-2

MINI PROJECT REPORT

TaskZen - Task Management and productivity tool

UCS2601 – INTERNET PROGRAMMING

**Department of Computer Science and Engineering
SSN College of Engineering**

**Semester and Section: VI Sem CSE A
Batch: 2023-27**



PRESENTED BY

Anushya V , 3122 23 5001 021(Developed the frontend interface, implemented core features such as boards, planner, and drag-and-drop task management, and integrated APIs for seamless user interaction.)

Faatina S, 3122 23 5001 046(Designed and implemented the backend system including database schema, REST APIs, authentication using JWT, and server-side logic for task and activity management.)

TABLE OF CONTENTS

1. Problem Statement

2. Objective

- 2.1 Task Organization
- 2.2 Deadline Management
- 2.3 Productivity Analytics
- 2.4 Notes Management
- 2.5 Centralized Planner
- 2.6 Activity Tracking
- 2.7 User Authentication
- 2.8 User Interface Design

3. System Analysis

- 3.1 Existing System
- 3.2 Proposed System
- 3.3 Feasibility Study

4. System Architecture

- 4.1 Presentation Layer (Frontend)
- 4.2 Application Layer (Backend)
- 4.3 Data Layer (Database)
- 4.4 Data Flow Architecture
- 4.5 Authentication Mechanism (JWT)
- 4.6 System Components

5. Workflow

- 5.1 User Authentication
- 5.2 Session Management and Authorization
- 5.3 Dashboard Initialization
- 5.4 Task Creation and Organization
- 5.5 Task Manipulation
- 5.6 Additional Functional Modules

- 5.7 Data Storage and Retrieval
- 5.8 Frontend–Backend Interaction
- 5.9 Logout and Session Termination

6. Technologies Used

7. Database Schema

8. Core Implementation (Code)

- 8.1 Frontend
- 8.2 Middleware
- 8.3 Backend
- 8.4 API Structure

9. Output

10. Future Enhancement

11. Learning Outcome

1. PROBLEM STATEMENT:

In today's fast-paced digital environment, individuals often struggle to manage tasks efficiently across multiple responsibilities such as academics, work, and personal life. Existing task management tools are either overly complex, lack intuitive design, or fail to provide a unified platform for organizing tasks, tracking deadlines, and visualizing progress.

As a result, users face:

- Difficulty in organizing tasks across different categories
- Poor visibility of deadlines and overdue tasks
- Lack of integration between task tracking, planning, and note-taking
- Reduced productivity due to cluttered or non-user-friendly interfaces

TaskZen aims to address these challenges by providing a simple, intuitive, and visually appealing platform that helps users manage tasks, track progress, and maintain productivity with ease.

2. OBJECTIVE:

2.1. Task Organization

- To enable users to create, update, and delete tasks efficiently
- To organize tasks into structured boards and columns (Todo, In Progress, Completed)
- To implement drag-and-drop functionality for seamless task movement

2.2. Deadline Management

- To allow users to assign due dates to tasks
- To automatically categorize tasks as Overdue, Due Today, or Upcoming
- To improve time management and task prioritization

2.3. Productivity Analytics

- To provide insights into task completion and progress
- To calculate and display completion rate
- To help users evaluate their productivity

2.4. Notes Management

- To allow users to capture ideas, reminders, and quick notes
- To implement color-coded notes for better organization
- To enhance quick access to important information

2.5. Centralized Planner

- To display all tasks from different boards in a single view
- To enable filtering of tasks based on their status
- To provide a comprehensive overview of all scheduled tasks

2.6. Activity Tracking

- To record user actions such as task creation, updates, and completion
- To maintain a history of activities for better tracking
- To improve transparency and workflow monitoring

2.7. User Authentication

- To implement secure login and signup using JWT
- To ensure that user data is protected and accessible only to authorized users
- To maintain session persistence across application usage

2.8. User Interface Design

- To design a clean, intuitive, and responsive user interface
- To use soft color palettes and modern layouts for better user experience
- To ensure smooth interaction and ease of use

3. SYSTEM ANALYSIS

3.1 Existing System

The existing task management approaches are mostly manual or use basic digital tools that lack integration and advanced features. Users often rely on notebooks, spreadsheets, or simple applications that do not provide real-time updates or structured organization. These systems do not support effective collaboration, making it difficult for multiple users to work together on shared tasks. Additionally, there is limited visualization of task progress, no intelligent filtering (such as overdue or upcoming tasks), and no centralized platform to manage all activities. As a result, productivity is reduced, and task tracking becomes inefficient and error-prone.

3.2 Proposed System

The proposed system, TaskZen, is a full-stack web-based task management application designed to improve productivity and organization. It provides a Kanban-style board where tasks can be easily created, updated, and moved between different stages such as To-Do, In Progress, and

Completed using drag-and-drop functionality. The system includes a planner module that categorizes tasks into overdue, due today, and upcoming, helping users prioritize their work effectively. Additional features such as notes, activity tracking, and analytics enhance usability. The system supports secure authentication using JWT and ensures seamless interaction between frontend and backend through RESTful APIs. Overall, it offers a centralized, user-friendly, and efficient solution for task management.

3.3 Feasibility Study

The proposed system is feasible from multiple perspectives.

Technical Feasibility: The system is built using widely used technologies such as React, Node.js, Express, and MySQL, which are reliable and scalable. The implementation does not require any specialized hardware or software beyond standard development tools.

Economic Feasibility: The development cost is minimal as it uses open-source technologies. Deployment on cloud platforms can also be achieved at low cost, making the system economically viable.

Operational Feasibility: The system is user-friendly and requires minimal training. Its intuitive interface ensures that users can easily manage tasks and navigate through features without difficulty.

Social Feasibility: The system promotes productivity and collaboration, making it suitable for students, professionals, and teams. It helps users organize their work efficiently and improves overall task management practices.

4. SYSTEM ARCHITECTURE:

4.1. Presentation Layer (Frontend)

The presentation layer of TaskZen is developed using React (with Vite) and is responsible for handling all user interactions and displaying the user interface. It includes components such as the Landing Page, Authentication Page, Boards, Planner, Notes, and Activity pages. This layer communicates with the backend through API calls using Axios. It ensures a smooth and responsive user experience by dynamically updating the UI based on user actions such as creating tasks, moving them across columns, or viewing analytics.

4.2. Application Layer (Backend)

The application layer is built using Node.js and Express.js, which handle the core business logic of the system. This layer processes incoming requests from the frontend, performs necessary validations, and interacts with the database. It includes multiple route modules such as authentication routes, task routes, board routes, checklist routes, notes routes, and activity routes. Additionally, it manages user authentication using JSON Web Tokens (JWT) and ensures secure communication between the client and the server.

4.3. Data Layer (Database)

The data layer consists of a relational database (such as MySQL) used to store and manage all application data. It maintains structured tables for users, boards, columns, tasks, checklist items, notes, and activity logs. This layer ensures data persistence and supports efficient querying and retrieval of information required by the application. It plays a crucial role in maintaining consistency and integrity of user data.

4.4. Data Flow Architecture

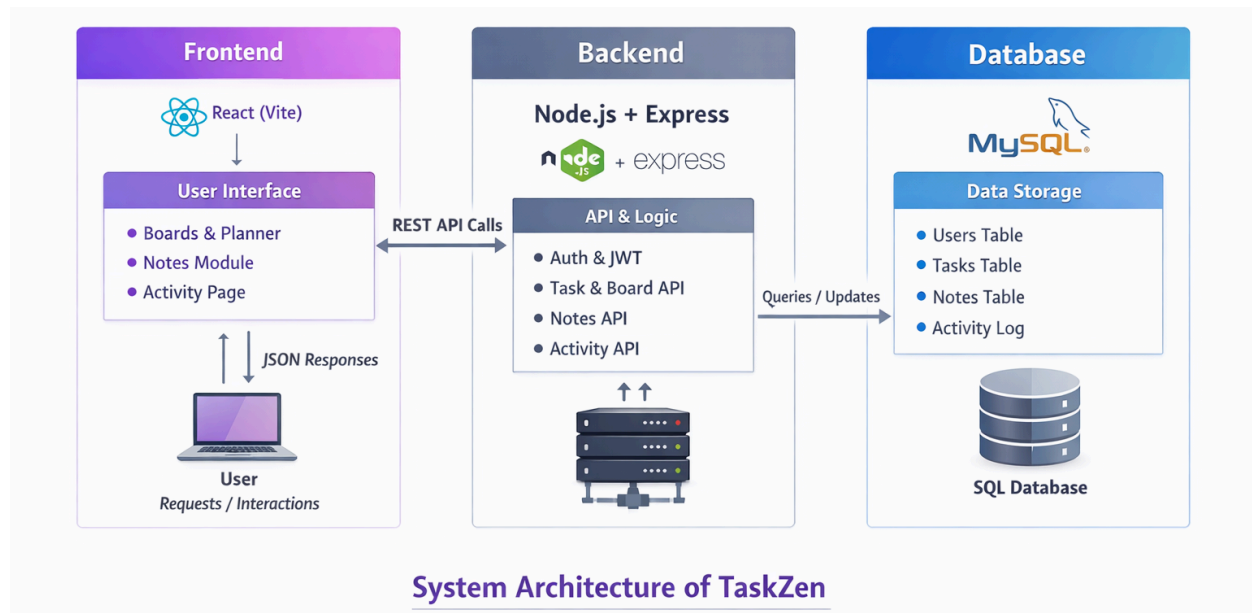
The data flow in TaskZen follows a client-server model. When a user performs an action, such as creating a task or logging in, the frontend sends a request to the backend via REST APIs. The backend processes the request, interacts with the database if required, and sends a response back to the frontend. The frontend then updates the user interface accordingly. This structured flow ensures smooth communication and real-time updates across the application.

4.5. Authentication Mechanism (JWT)

TaskZen implements secure user authentication using JSON Web Tokens (JWT). When a user logs in, the backend verifies the credentials and generates a token, which is stored in the browser's local storage. This token is included in the headers of subsequent API requests to authenticate the user. The backend validates the token before granting access to protected routes, ensuring that only authorized users can access their data.

4.6. System Components

The system is composed of multiple interconnected components across the frontend and backend. The frontend includes modules such as task boards, planner, notes, and activity tracking interfaces. The backend consists of route handlers and controllers for managing different functionalities. These components work together to provide a seamless experience, enabling users to manage tasks, track progress, and monitor activities efficiently.



5. WORKFLOW:

5.1 User Authentication

The workflow begins when the user accesses the application and either registers as a new user or logs in using existing credentials. The user provides basic details like name, email and password during registration. For login, the entered credentials are verified by the backend server. Upon successful authentication, a JSON Web Token (JWT) is generated and sent to the client which is stored on the client side and used to maintain the user session securely.

5.2 Session Management and Authorization

Once the user is authenticated, the JWT token is included in the header of every subsequent request made to protected endpoints. The backend middleware verifies the token to ensure that the request is coming from an authorized user. If the token is valid, access is granted; otherwise, the request is rejected. This mechanism ensures that only authenticated users can access their respective data.

5.3 Dashboard Initialization

After successful login, the user is redirected to the main dashboard. At this stage, the frontend sends requests to the backend APIs to fetch user-specific data such as tasks, boards, and activity logs. The backend retrieves the required information from the database and sends it back to the client in JSON format. The frontend then dynamically renders this data, providing the user with an interactive interface.

5.4 Task Creation and Organization

Users can create new tasks by entering relevant details such as title, description, and due date. When a task is created, the frontend sends a request to the backend, which processes the request and stores the task in the database associated with the logged-in user. Tasks are organized into different stages based on their progress, allowing users to manage their work efficiently.

5.5 Task Manipulation

The application allows users to perform various operations on tasks, including updating task details, moving tasks between stages, and deleting tasks. These actions trigger appropriate API requests such as PUT and DELETE. The backend updates the database accordingly, and the frontend reflects these changes instantly, ensuring a seamless user experience.

5.6 Additional Functional Modules

The system includes additional modules to enhance usability and user experience. The Notes module allows users to quickly capture ideas, reminders, or important information in a flexible and visually organized manner. Users can assign colors to notes for better categorization and identification. The Planner module helps users view and manage tasks based on deadlines, improving time management. The Activity module records all user actions, such as task creation, updates, and deletions, enabling users to track their activity history effectively and also a Analytics Dashboard in the boards Section to visualize the tasks.

5.7 Data Storage and Retrieval

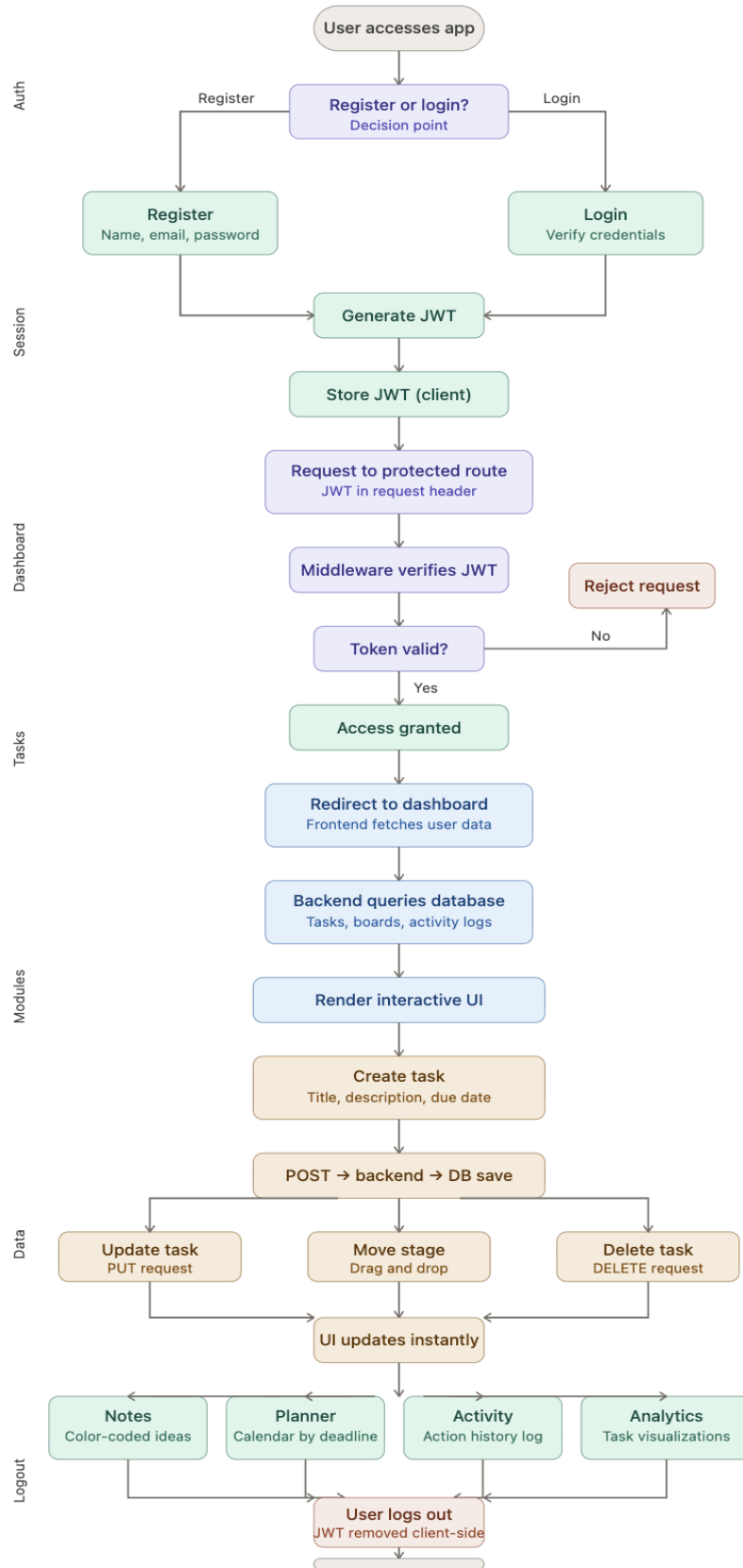
All data related to users, tasks, and activities is stored in a MySQL database. The backend is responsible for handling all CRUD operations. Data retrieval is always performed based on the authenticated user, ensuring privacy and data isolation.

5.8 Frontend–Backend Interaction

The frontend communicates with the backend through RESTful APIs using HTTP methods such as GET, POST, PUT, and DELETE. The backend processes these requests and returns responses in JSON format. The frontend uses this data to update the user interface dynamically without requiring a page reload.

5.9 Logout and Session Termination

When the user logs out, the JWT token is removed from the client side, effectively terminating the session. This ensures that unauthorized access is prevented unless the user logs in again.



6. TECHNOLOGIES USED:

- **Frontend:** React.js (Vite)
- **Styling:** CSS (inline styling)
- **Backend:** Node.js
- **Framework:** Express.js
- **Database:** MySQL
- **API Communication:** REST APIs
- **HTTP Client:** Axios
- **Authentication:** JWT (JSON Web Token)
- **State Management:** React Hooks (useState, useEffect, Context API)
- **Drag & Drop:** @hello-pangea/dnd
- **Development Tools:** VS Code
- **Package Manager:** npm
- **Version Control:** Git

7. DATABASE SCHEMA:

```
CREATE DATABASE task_db;
```

```
USE task_db;
```

```
CREATE TABLE IF NOT EXISTS users (  
  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  
  name VARCHAR(255) NOT NULL,  
  
  email VARCHAR(255) NOT NULL UNIQUE,  
  
  password VARCHAR(255) NOT NULL,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
  
);
```

```
CREATE TABLE IF NOT EXISTS boards (  
  
  id INT AUTO_INCREMENT PRIMARY KEY,
```

```
title VARCHAR(255) NOT NULL,  
  
user_id INT NOT NULL,  
  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
  
);
```

```
CREATE TABLE IF NOT EXISTS `columns` (  
  
id INT AUTO_INCREMENT PRIMARY KEY,  
  
title VARCHAR(255) NOT NULL,  
  
board_id INT NOT NULL,  
  
position INT DEFAULT 0,  
  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
FOREIGN KEY (board_id) REFERENCES boards(id) ON DELETE CASCADE  
  
);
```

```
CREATE TABLE IF NOT EXISTS tasks (  
  
id INT AUTO_INCREMENT PRIMARY KEY,  
  
title VARCHAR(255) NOT NULL,  
  
description TEXT,  
  
column_id INT DEFAULT NULL,  
  
user_id INT NOT NULL,  
  
position INT DEFAULT 0,  
  
due_date DATE DEFAULT NULL,  
  
is_inbox TINYINT(1) DEFAULT 0,
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    FOREIGN KEY (column_id) REFERENCES `columns` (id) ON DELETE CASCADE,  
  
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

```
CREATE TABLE IF NOT EXISTS checklist (  
  
    id INT AUTO_INCREMENT PRIMARY KEY,  
  
    text VARCHAR(255) NOT NULL,  
  
    task_id INT NOT NULL,  
  
    is_completed TINYINT(1) DEFAULT 0,  
  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    FOREIGN KEY (task_id) REFERENCES tasks(id) ON DELETE CASCADE  
);
```

```
CREATE TABLE IF NOT EXISTS activity (  
  
    id INT AUTO_INCREMENT PRIMARY KEY,  
  
    message VARCHAR(500) NOT NULL,  
  
    task_id INT DEFAULT NULL,  
  
    user_id INT DEFAULT NULL,  
  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE IF NOT EXISTS notes (  
  
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
text          VARCHAR(500) NOT NULL,

color         VARCHAR(20) DEFAULT '#ffd93d',

user_id       INT NOT NULL,

created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE

);
```

8. CORE IMPLEMENTATION (CODE):

8.1 Frontend

api.js

```
import axios from "axios";

const API = axios.create({
  baseURL: "http://localhost:5000/api",
});

API.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");

  if (token) config.headers.Authorization = `Bearer ${token}`;

  return config;
});

export default API;
```

AuthPage.jsx

```
import { useState } from "react";
import API from "../services/api";
import { useAuth } from "../context/AuthContext";
import { useNavigate } from "react-router-dom";
```

```

function AuthPage() {
  const { login } = useAuth();
  const navigate = useNavigate();
  const [mode, setMode] = useState("login");
  const [form, setForm] = useState({ name: "", email: "", password: "" });
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  // Handle input changes
  const handle = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };
  // Submit authentication
  const submit = async () => {
    setError("");
    setLoading(true);
    try {
      // Choose endpoint
      const endpoint =
        mode === "login" ? "/auth/login" : "/auth/signup";
      // Prepare payload
      const payload =
        mode === "login"
          ? { email: form.email, password: form.password }
          : { name: form.name, email: form.email, password: form.password };
      // API call
      const res = await API.post(endpoint, payload);
      // Store token + user
      login(res.data.token, res.data.user);
      // Redirect to dashboard
      navigate("/boards", { replace: true });
    } catch (err) {
      setError(
        err.response?.data?.message ||
        err.message ||
        "Server not reachable"
      );
    } finally {
      setLoading(false);
    }
  };
}

```

```

    }
  };
  return null;
}
export default AuthPage;

```

BoardPage.jsx

```

import { useEffect, useState } from "react"; import API from "../services/api";

function BoardPage({ id }) {
  const [columns, setColumns] = useState([]);
  useEffect(() => { fetchAll(); }, [id]);
  const fetchAll = async () => {
    const colsRes = await API.get(`/columns/${id}`);
    const cols = colsRes.data;
    const taskResults = await Promise.all(cols.map(col =>
    API.get(`/tasks/${col.id}`)));
    const enriched = cols.map((col,i) => ({ ...col, tasks: taskResults[i].data }));
    setColumns(enriched);
  };

  const handleDragEnd = (result) => {
    const { source, destination, draggableId } = result;
    if (!destination) return;
    const sourceCol = columns.find(c => c.id === parseInt(source.droppableId));
    const destCol = columns.find(c => c.id === parseInt(destination.droppableId));
    const taskIndex = sourceCol.tasks.findIndex(t => t.id === parseInt(draggableId));
    const [task] = sourceCol.tasks.splice(taskIndex,1);
    task.column_id = destCol.id;
    destCol.tasks.splice(destination.index,0,task);
    setColumns([...columns]);
    API.put(`/tasks/move/${task.id}`, { column_id: destCol.id });
  };

  const addTask = async (colId,title,dueDate) => {
    if (!title.trim()) return;

```

```

}); await API.post("/tasks/create",{ title, column_id: colId, due_date: dueDate || null
    fetchAll();
});

const deleteTask = async (taskId) => {
    await API.delete(`/tasks/${taskId}`);
    fetchAll();
};

const getStatus = (task,columnTitle) => {
    const today = new Date(); today.setHours(0,0,0,0);
    const due = task.due_date ? new Date(task.due_date) : null;
    if (due) due.setHours(0,0,0,0);
    const isCompleted = columnTitle === "Completed";
    if (due && !isCompleted) {
        if (due < today) return "Overdue";
        if (due.getTime() === today.getTime()) return "Due Today";
        return "Upcoming";
    }
    return "";
};

return null;
}

export default BoardPage;

```

Planner.jsx

```

import { useEffect, useState } from "react";
import API from "../services/api";

function Planner() {
    const [tasks, setTasks] = useState([]);
    const [filter, setFilter] = useState("all");
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        fetchTasks();
    });

```

```

}, []);

const fetchTasks = async () => {
  try {
    const res = await API.get("/tasks/all");
    setTasks(res.data);
  } catch (err) {
    console.error(err);
  } finally {
    setLoading(false);
  }
};

const today = new Date();
today.setHours(0, 0, 0, 0);

const getStatus = (task) => {
  if (task.column_title === "Completed") return "none";
  if (!task.due_date) return "none";

  const d = new Date(task.due_date);
  d.setHours(0, 0, 0, 0);

  if (d < today) return "overdue";
  if (d.getTime() === today.getTime()) return "today";
  return "upcoming";
};

const filtered = tasks.filter(t =>
  filter === "all" ? true : getStatus(t) === filter
);

const counts = {
  overdue: tasks.filter(t => getStatus(t) === "overdue").length,
  today: tasks.filter(t => getStatus(t) === "today").length,
  upcoming: tasks.filter(t => getStatus(t) === "upcoming").length
};

return null;}export default Planner;

```

Boards.jsx

```
import { useEffect, useState } from "react";
import API from "../services/api";

function Boards() {
  const [boards, setBoards] = useState([]);
  const [title, setTitle] = useState("");
  const [adding, setAdding] = useState(false);

  useEffect(() => {
    fetchBoards();
  }, []);

  const fetchBoards = async () => {
    try {
      const res = await API.get("/boards");
      setBoards(res.data);
    } catch (err) {
      console.error(err);
    }
  };

  const createBoard = async () => {
    if (!title.trim()) return;

    try {
      await API.post("/boards/create", { title });
      setTitle("");
      setAdding(false);
      fetchBoards();
    } catch (err) {
      console.error(err);
    }
  };

  const deleteBoard = async (id) => {
    try {
      await API.delete(`/boards/${id}`);
    }
  };
}
```

```

        fetchBoards();
    } catch (err) {
        console.error(err);
    }
};

return null;
}

export default Boards;

```

8.2 MiddleWare

auth.js

```

const jwt = require("jsonwebtoken");
module.exports = (req, res, next) => {
    const authHeader = req.headers["authorization"];
    const token = authHeader && authHeader.split(" ")[1]; // Bearer <token>
    if (!token) return res.status(401).json({ message: "No token, access denied" });
    try {
        const decoded = jwt.verify(token, process.env.JWT_SECRET);
        req.user = decoded; }
    next();
    } catch (err) {
        res.status(401).json({ message: "Invalid or expired token" });
    }
};

```

8.3 Backend

server.js

```

const express = require("express");
const cors = require("cors");
require("dotenv").config();

const db = require("./config/db");
const authRoutes = require("./routes/authRoutes");
const boardRoutes = require("./routes/boardRoutes");

```

```

const columnRoutes = require("./routes/columnRoutes");
const taskRoutes = require("./routes/taskRoutes");
const checklistRoutes = require("./routes/checklistRoutes");
const activityRoutes = require("./routes/activityRoutes");
const notesRoutes = require("./routes/notesRoutes");

const app = express();
app.use(cors());
app.use(express.json());

app.use("/api/auth", authRoutes);
app.use("/api/boards", boardRoutes);
app.use("/api/columns", columnRoutes);
app.use("/api/tasks", taskRoutes);
app.use("/api/checklist", checklistRoutes);
app.use("/api/activity", activityRoutes);
app.use("/api/notes", notesRoutes);

app.get("/", (req, res) => res.send("Focusly server running"));
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));

```

DataBase Connection

db.js

```

const mysql = require("mysql2");
require("dotenv").config();

const db = mysql.createConnection({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
});

db.connect((err) => {
  if (err) {
    console.error("DB connection failed:", err);
  }
});

```

```

    } else {
      console.log("Connected to MySQL");
    }
  });
};

```

```
module.exports = db;
```

AuthController.js

```

const db = require("../config/db");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");

// SIGNUP
exports.signup = (req, res) => {
  const { name, email, password } = req.body;

  if (!name || !email || !password)
    return res.status(400).json({ message: "All fields are required" });

  // Check if email already exists
  db.query("SELECT id FROM users WHERE email = ?", [email], (err, rows) => {
    if (err) return res.status(500).json(err);
    if (rows.length > 0)
      return res.status(409).json({ message: "Email already registered" });

    const hashed = bcrypt.hashSync(password, 10);

    db.query(
      "INSERT INTO users (name, email, password) VALUES (?, ?, ?)",
      [name, email, hashed],
      (err, result) => {
        if (err) return res.status(500).json(err);

        const userId = result.insertId;
        const token = jwt.sign(
          { id: userId, name, email },
          process.env.JWT_SECRET,

```

```

        { expiresIn: "7d" }
    );

    res.json({ token, user: { id: userId, name, email } });
}
});
});

// LOGIN
exports.login = (req, res) => {
    const { email, password } = req.body;

    if (!email || !password)
        return res.status(400).json({ message: "Email and password required" });

    db.query("SELECT * FROM users WHERE email = ?", [email], (err, rows) => {
        if (err) return res.status(500).json(err);
        if (rows.length === 0)
            return res.status(401).json({ message: "Invalid email or password" });

        const user = rows[0];
        const match = bcrypt.compareSync(password, user.password);

        if (!match)
            return res.status(401).json({ message: "Invalid email or password" });

        const token = jwt.sign(
            { id: user.id, name: user.name, email: user.email },
            process.env.JWT_SECRET,
            { expiresIn: "7d" }
        );

        res.json({ token, user: { id: user.id, name: user.name, email: user.email } });
    });
};

// GET current user info

```

```

exports.getMe = (req, res) => {
  db.query(
    "SELECT id, name, email, created_at FROM users WHERE id = ?",
    [req.user.id],
    (err, rows) => {
      if (err) return res.status(500).json(err);
      if (rows.length === 0) return res.status(404).json({ message: "User not found" });
      res.json(rows[0]);
    }
  );
};

```

boardController.js

```

const db = require("../config/db");
// Create Board - auto-creates 3 fixed columns
exports.createBoard = (req, res) => {
  const { title } = req.body;
  const userId = req.user.id;
  if (!title) return res.status(400).json({ message: "Title required" });
  db.query(
    "INSERT INTO boards (title, user_id) VALUES (?, ?)",
    [title, userId],
    (err, result) => {
      if (err) return res.status(500).json(err);
      const boardId = result.insertId;
      // Auto-create 3 fixed columns
      const cols = [
        ["Todo", boardId, 1],
        ["In Progress", boardId, 2],
        ["Completed", boardId, 3],
      ];
      db.query(
        "INSERT INTO `columns` (title, board_id, position) VALUES ?",
        [cols],
        (err2) => {
          if (err2) return res.status(500).json(err2);
          res.json({ message: "Board created", id: boardId });
        }
      );
    }
  );
};

```

```

    }
  );
}
);
};

// Get All Boards for logged-in user
exports.getBoards = (req, res) => {
  const userId = req.user.id;
  db.query(
    "SELECT * FROM boards WHERE user_id = ? ORDER BY created_at DESC",
    [userId],
    (err, result) => {
      if (err) return res.status(500).json(err);
      res.json(result);
    }
  );
};

// Delete Board
exports.deleteBoard = (req, res) => {
  const { id } = req.params;
  const userId = req.user.id;
  db.query(
    "DELETE FROM boards WHERE id = ? AND user_id = ?",
    [id, userId],
    (err) => {
      if (err) return res.status(500).json(err);
      res.json({ message: "Board deleted" });
    }
  );
};
};

```

taskController.js

```

const db = require("../config/db");

// Create Task
exports.createTask = (req, res) => {
  let { title, description = "", column_id, due_date, is_inbox = 0 } = req.body;
  const userId = req.user.id;
  if (!title) return res.status(400).json({ message: "Title is required" });

```

```

    if (!is_inbox && !column_id)
        return res.status(400).json({ message: "column_id required for board tasks" });
    if (!due_date || due_date === "") due_date = null;
    const finalColumnId = is_inbox ? null : column_id;
    db.query(
        "INSERT INTO tasks (title, description, column_id, user_id, position, due_date, is_inbox) VALUES (?, ?, ?, ?, ?, ?, ?)",
        [title, description, finalColumnId, userId, 0, due_date, is_inbox ? 1 : 0],
        (err, result) => {
            if (err) { console.error(err); return res.status(500).json(err); }
            const taskId = result.insertId;
            db.query(
                "INSERT INTO activity (message, task_id, user_id) VALUES (?, ?, ?)",
                [`Task created: ${title}`, taskId, userId]
            );
            res.json({ message: "Task created", id: taskId });
        }
    );
};

// Get Tasks by Column
exports.getTasks = (req, res) => {
    const columnId = req.params.columnId;
    db.query(
        "SELECT * FROM tasks WHERE column_id = ? ORDER BY position ASC",
        [columnId],
        (err, result) => {
            if (err) return res.status(500).json(err);
            res.json(result);
        }
    );
};

// All tasks with due dates (Planner) - scoped to user
exports.getAllTasksWithDates = (req, res) => {
    const userId = req.user.id;
    db.query(
        `SELECT t.*, c.title AS column_title, b.title AS board_title
        FROM tasks t
    `
    );
};

```

```

    LEFT JOIN \`columns\` c ON t.column_id = c.id
    LEFT JOIN boards b ON c.board_id = b.id
    WHERE t.due_date IS NOT NULL AND t.user_id = ?
    ORDER BY t.due_date ASC`,
    [userId],
    (err, result) => {
      if (err) return res.status(500).json(err);
      res.json(result);
    }
  );
};

// Move Task
exports.moveTask = (req, res) => {
  const { taskId } = req.params;
  const { column_id } = req.body;
  const userId = req.user.id;
  {db.query("UPDATE tasks SET column_id = ? WHERE id = ?", [column_id, taskId], (err) => {
    if (err) return res.status(500).json(err);
    db.query(
      "INSERT INTO activity (message, task_id, user_id) VALUES (?, ?, ?)",
      [`Task moved to column ${column_id}`, taskId, userId]
    );
    res.json({ message: "Task moved successfully" });});});
};

// Reorder tasks
exports.reorderTasks = (req, res) => {
  const { tasks } = req.body;
  const updates = tasks.map((task, index) =>
    new Promise((resolve, reject) => {
      (err) => {db.query("UPDATE tasks SET position = ? WHERE id = ?", [index + 1, task.id],
        if (err) reject(err); else resolve();}); });
    Promise.all(updates)
      .then(() => res.json({ message: "Tasks reordered" }))
      .catch((err) => res.status(500).json(err));
  });
};

// Delete Task
exports.deleteTask = (req, res) => {
  const { id } = req.params;

```

```

const userId = req.user.id;
db.query("DELETE FROM tasks WHERE id = ?", [id], (err) => {
  if (err) return res.status(500).json(err);
  db.query(
    "INSERT INTO activity (message, task_id, user_id) VALUES (?, ?, ?)",
    ["Task deleted", id, userId]
  );
  res.json({ message: "Task deleted" });
});
};

```

8.4 Api Structure

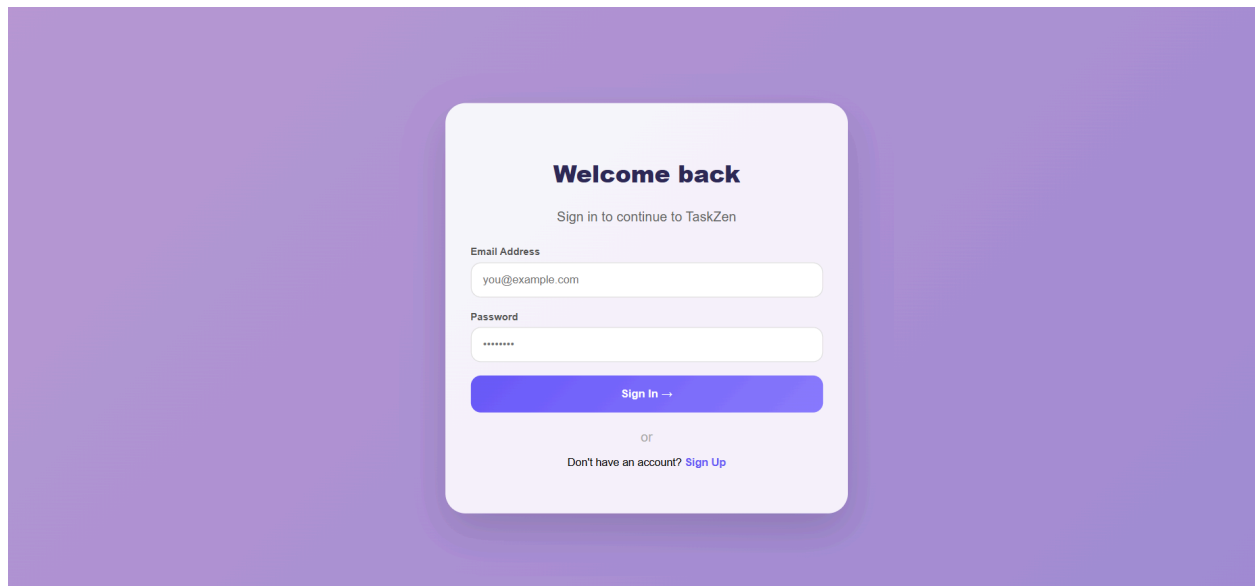
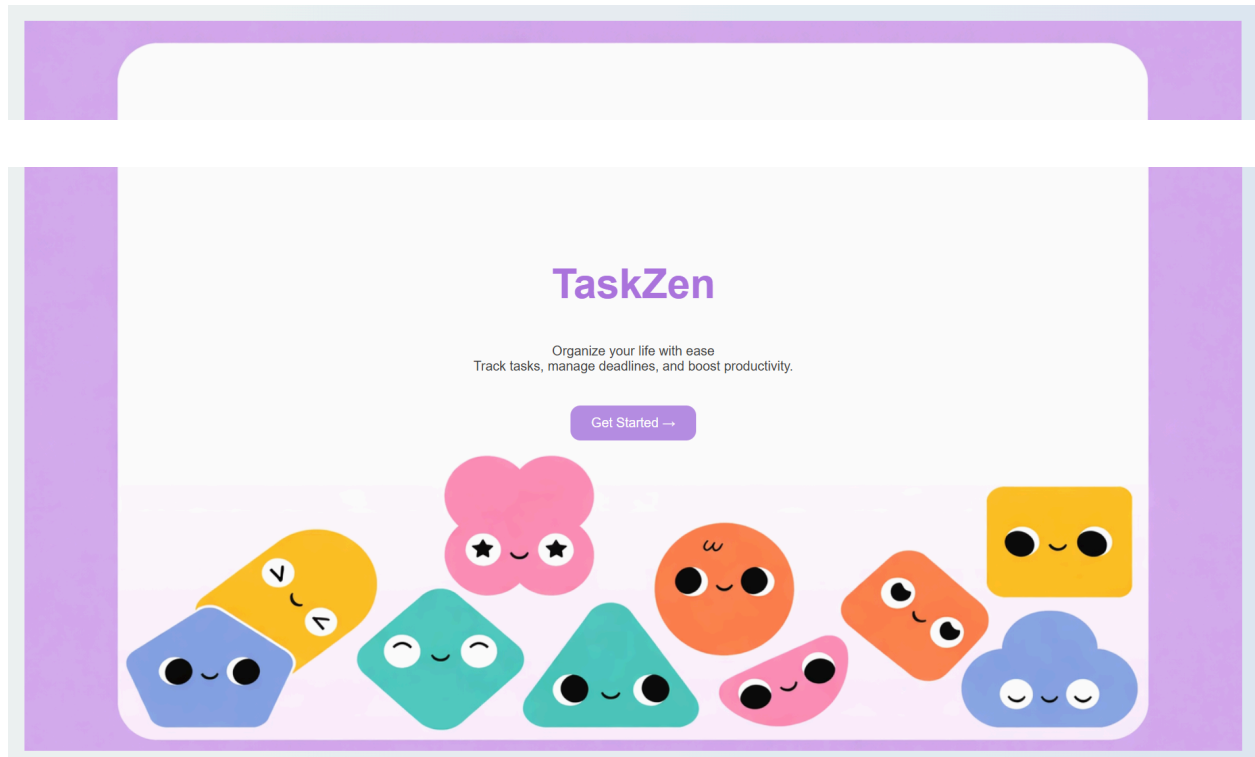
taskRoutes.js

```

const express = require("express")
const router = express.Router();
const db = require("../config/db");
const auth = require("../middleware/auth");
const {
  createTask, getTasks, moveTask,
  reorderTasks, deleteTask, getAllTasksWithDates,
} = require("../controllers/taskController");
router.post("/create", auth, createTask);
router.get("/inbox", auth, (req, res) => {
  db.query(
    "SELECT * FROM tasks WHERE is_inbox = TRUE AND user_id = ? ORDER BY created_at DESC",
    [req.user.id],
    (err, rows) => {
      if (err) return res.status(500).json(err);
      res.json(rows);
    });
});
router.get("/all", auth, getAllTasksWithDates);
router.get("/:columnId", getTasks);
router.put("/move/:taskId", auth, moveTask);
router.put("/reorder", auth, reorderTasks);
router.delete("/:id", auth, deleteTask);
module.exports = router;

```

9. OUTPUT:



Welcome back

Sign in to continue to TaskZen

Email Address

anu123@gmail.com

Password

⚠ Invalid email or password

Sign In →

OR

Don't have an account? [Sign Up](#)

Get started 🚀

Create your free TaskZen account

Full Name

Anushya

Email Address

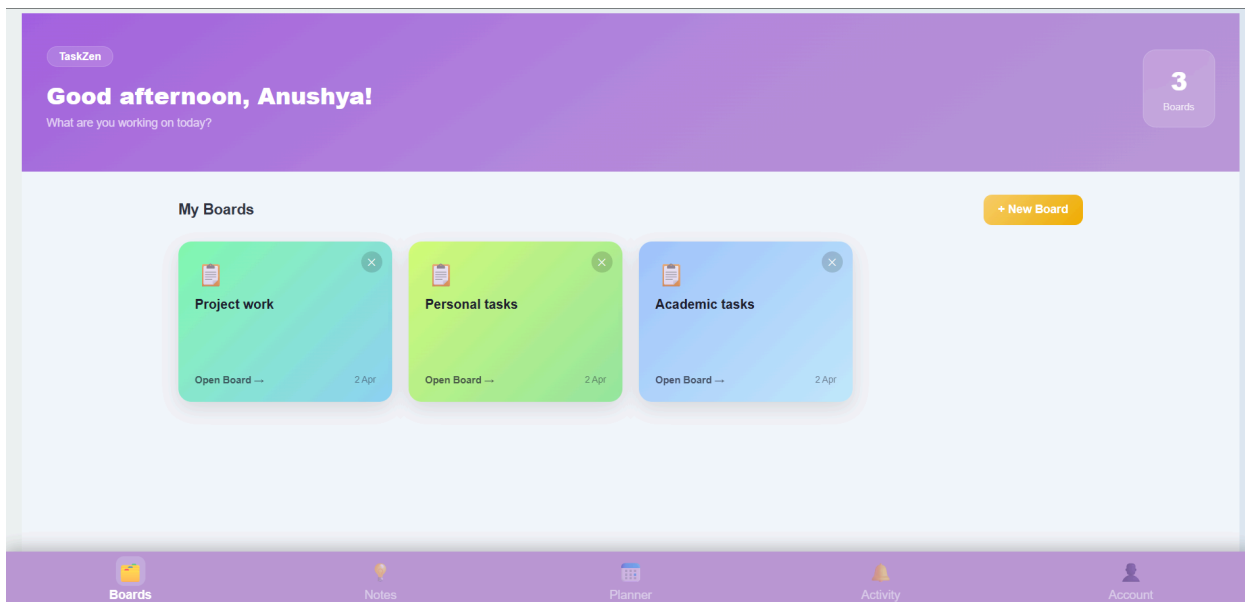
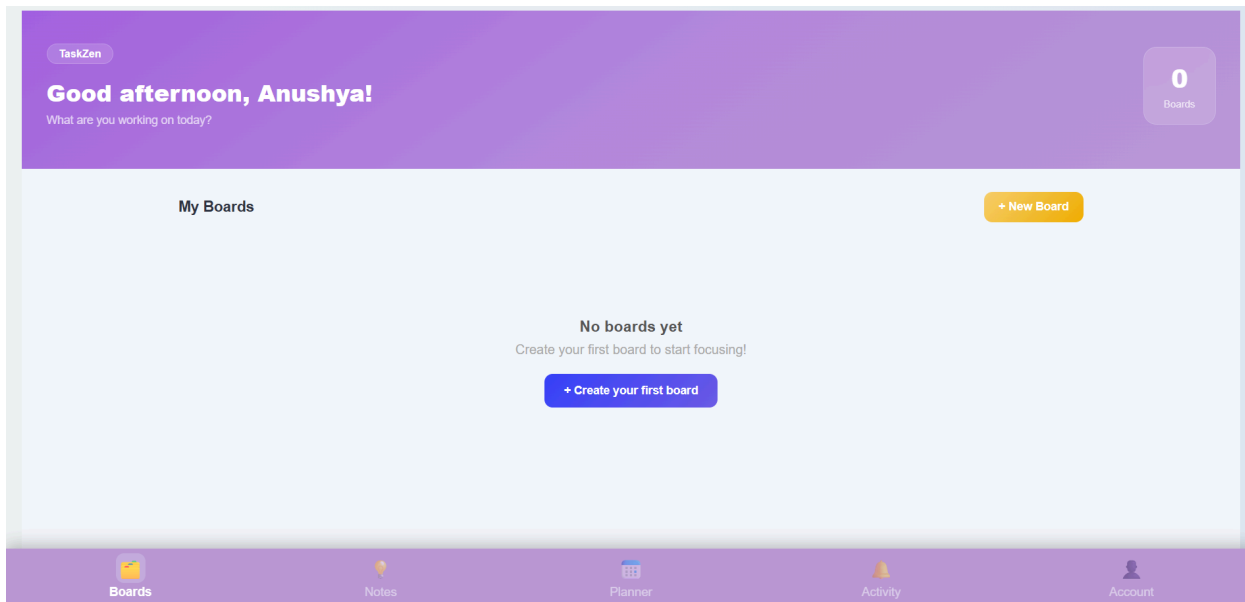
anushyavinayagam@gmail.com

Password

Create Account →

OR

Already have an account? [Sign In](#)



← Back

Academic tasks

0

Todo

Add a task...

dd-mm-yyyy --:--

+ Add Task

0

In Progress

Add a task...

dd-mm-yyyy --:--

+ Add Task

0

Completed

Add a task...

dd-mm-yyyy --:--

+ Add Task

Analytics Dashboard

Total Tasks

0

Completed

0

Completion

0%

Boards

Notes

Planner

Activity

Account

Analytics Dashboard

Total Tasks

0

Completed

0

Completion

0%

Tasks Over Time

Boards

Notes

Planner

Activity

Account

← Back

Academic tasks

Todo2

Submit assignment

6 Apr · Upcoming

Show Checklist ▼

Prepare for exam

9 Apr · Upcoming

Show Checklist ▼

Add a task...

dd-mm-yyyy --:--

+ Add Task

In Progress2

Complete Record

2 Apr · Due Today

Show Checklist ▼

Submit Observation

2 Apr · Due Today

Show Checklist ▼

Add a task...

dd-mm-yyyy --:--

+ Add Task

Completed1

Project submission

9 Apr

Show Checklist ▼

Add a task...

dd-mm-yyyy --:--

+ Add Task

Analytics Dashboard

Total Tasks

5

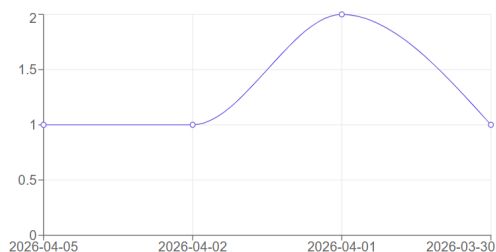
Completed

1

Completion



Tasks Over Time



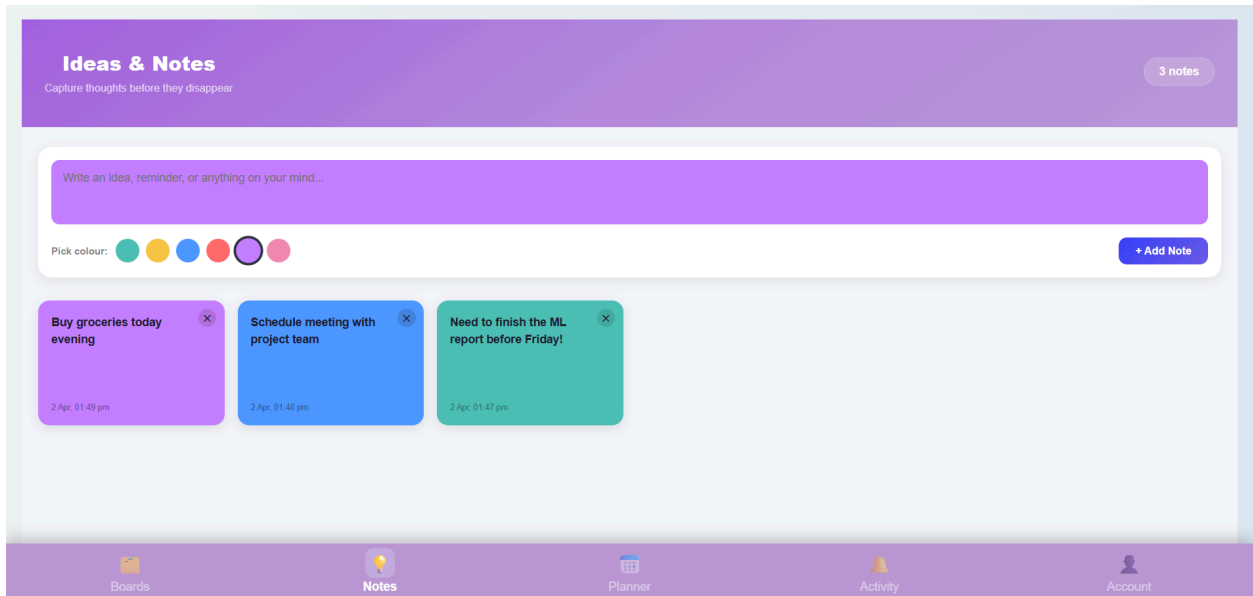
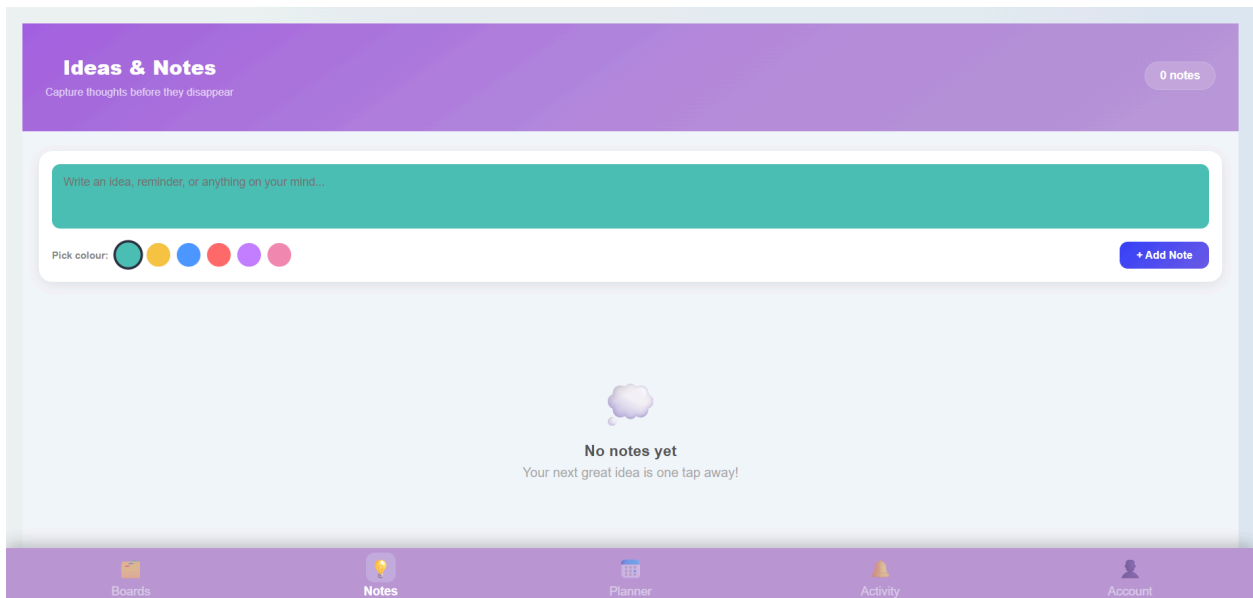
Boards

Notes

Planner

Activity

Account



Planner

All tasks with due dates across your boards

0

Overdue

2

Due Today

2

Upcoming

All

Overdue

Today

Upcoming

Project submission

Academic tasks > Completed

No Date
31 Mar 2026

Complete Record

Academic tasks > In Progress

Due Today
2 Apr 2026

Submit Observation

Academic tasks > In Progress

Due Today
2 Apr 2026

Prepare for exam

Academic tasks > Todo

Upcoming
3 Apr 2026

Boards

Notes

Planner

Activity

Account

0

Overdue

2

Due Today

2

Upcoming

All

Overdue

Today

Upcoming

Project submission

Academic tasks > Completed

No Date
31 Mar 2026

Complete Record

Academic tasks > In Progress

Due Today
2 Apr 2026

Submit Observation

Academic tasks > In Progress

Due Today
2 Apr 2026

Prepare for exam

Academic tasks > Todo

Upcoming
3 Apr 2026

Submit assignment

Academic tasks > Todo

Upcoming
6 Apr 2026

Boards

Notes

Planner

Activity

Account

Recent actions on your tasks

Clear All

- | | | |
|---|---|-----------------|
|  | Task created: Submit Observation
Task: Submit Observation | 2 Apr, 01:45 pm |
|  | Task created: Project submission
Task: Project submission | 2 Apr, 01:43 pm |
|  | Task created: Complete Record
Task: Complete Record | 2 Apr, 01:42 pm |
|  | Task created: Prepare for exam
Task: Prepare for exam | 2 Apr, 01:41 pm |
|  | Task created: Submit assignment
Task: Submit assignment | 2 Apr, 01:41 pm |
|  | Task deleted | 2 Apr, 01:40 pm |
|  | Task created: Submit Assignment | 2 Apr, 01:40 pm |



Activity

Account

A

Anushya

YOUR ACTIVITY

3 Boards

6

Tasks

3

Notes

Sign Out

Account

10. FUTURE ENHANCEMENT

To deploy the application on cloud platforms

The system can be deployed on cloud platforms such as AWS, Google Cloud, or Vercel to make it accessible from anywhere. This will improve scalability, reliability, and allow real-time usage beyond local environments.

To enable multi-user collaboration

The application can be enhanced to support multiple users working on the same board simultaneously. Features like shared boards, role-based access (admin/member), and real-time updates will improve team productivity and collaboration.

To develop a mobile application

A mobile version of the application can be built using React Native or Flutter to provide users with easy access on smartphones and tablets.

To integrate AI-based task recommendations

Machine learning techniques can be used to suggest task priorities, deadlines, or productivity insights based on user behavior.

11. LEARNING OUTCOME:

- Gained practical experience in building a complete application using frontend, backend, and database technologies.
- Learned how to implement user authentication and authorization using JSON Web Tokens (JWT) to ensure data security.
- Developed and integrated RESTful APIs for efficient communication between client and server.
- Gained knowledge in designing relational databases and performing CRUD operations using MySQL.
- Built a responsive and dynamic user interface with features like task organization, note taking, and real time updates.
- Improved ability to design scalable systems, debug issues, and implement features that enhance user productivity.